SAECOIN

BLOCKCHAIN ARCHITECTURE

Document Version: 1.0
Project: SAECOIN
Ticker: SAC (Provisional)
Blockchain Type: Independent Layer-1
Consensus: Proof of Work (PoW)
Architecture: Modular Decentralized Blockchain
Document Status: Planning

---

1. PURPOSE

This document defines the overall software and network architecture of SAECOIN.

SAECOIN will operate as an independent decentralized blockchain network.

It will not depend on Bitcoin, Ethereum, BNB Chain, or another blockchain to process its native transactions.

The architecture must support:

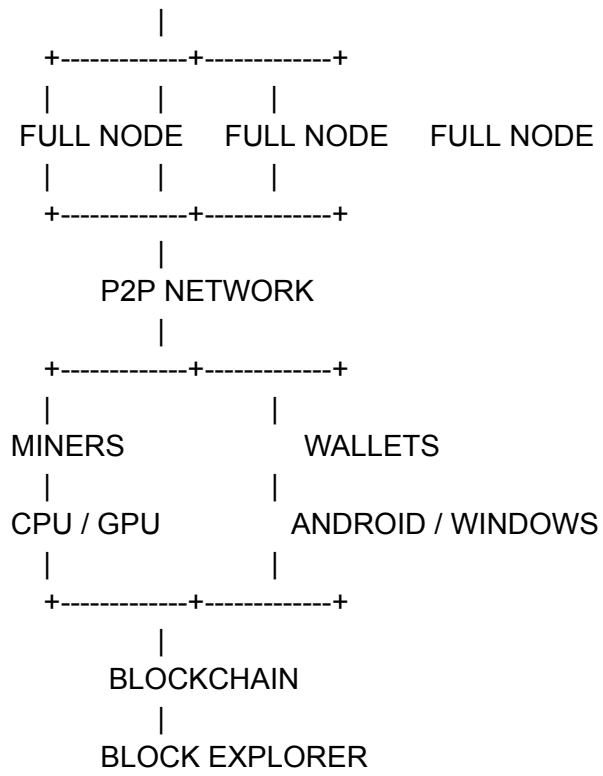
- Independent blockchain operation.
- Decentralized node communication.
- Proof-of-Work consensus.
- CPU and GPU mining.
- UTXO transaction processing.
- Android and Windows wallets.
- Block explorer.
- Future mining pool integration.
- Network expansion.

---

2. HIGH-LEVEL ARCHITECTURE

The SAECOIN ecosystem will consist of the following major components:

SAECOIN NETWORK



This is a logical overview. Actual network connections will be peer-to-peer, and no single central server should control consensus.

---

3. MODULAR SOFTWARE ARCHITECTURE

The codebase should be divided into independent modules.

3.1 Blockchain Core

Responsibilities:

- Block structure.
- Transaction structure.
- Cryptographic operations.
- Block hashing.
- Transaction validation.
- UTXO management.
- Chain state management.

This is the foundation of SAECOIN.

3.2 Consensus Engine

Responsibilities:

- Proof-of-Work validation.
- Difficulty calculation.
- Block reward calculation.
- Halving.
- Block validity rules.
- Chain selection.

Consensus rules must be identical across all compatible nodes.

3.3 Storage Engine

Responsibilities:

- Block storage.
- Transaction storage.
- UTXO storage.
- Block indexing.
- Undo data.
- Chain metadata.

The initial implementation may use RocksDB, subject to benchmarking.

3.4 P2P Network Module

Responsibilities:

- Peer discovery.
- Peer connections.
- Message transmission.
- Block propagation.
- Transaction propagation.
- Blockchain synchronization.

3.5 Mining Module

Responsibilities:

- Mining job creation.
- CPU mining.
- GPU mining.
- Nonce searching.
- Proof submission.
- Mining statistics.

Mining software must not be trusted to validate its own blocks. Nodes independently verify every submitted block.

3.6 Wallet Core

Responsibilities:

- Key generation.
- Address generation.
- Transaction construction.
- Transaction signing.
- Balance calculation.
- Transaction verification.

3.7 Explorer Module

Responsibilities:

- Block lookup.
- Transaction lookup.
- Address lookup.
- Network statistics.
- Mining statistics.

Explorer data will be derived from validated blockchain data.

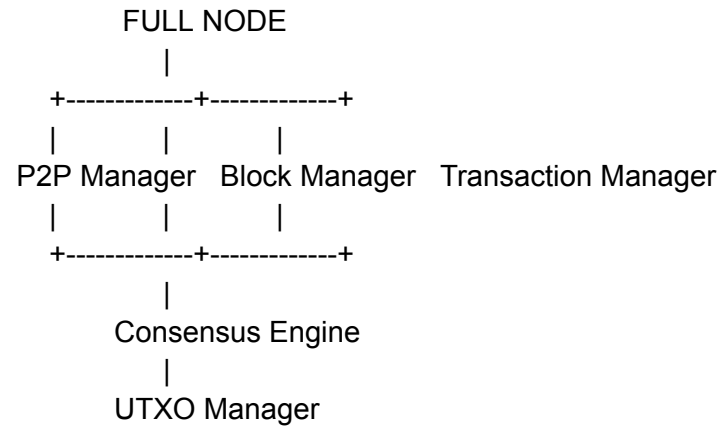
---

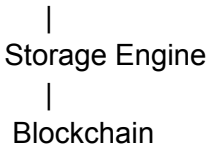
4. FULL NODE ARCHITECTURE

A full node is an independent participant in the SAECOIN network.

Every full node must independently validate the blockchain according to the consensus rules.

Node Components





Node Responsibilities

1. Connect to other nodes.
2. Discover peers.
3. Receive blocks.
4. Validate blocks.
5. Validate transactions.
6. Maintain the UTXO set.
7. Relay valid transactions.
8. Relay valid blocks.
9. Synchronize blockchain data.
10. Reject invalid data.

A node must never accept a block simply because another peer claims it is valid.

---

5. PEER-TO-PEER NETWORK ARCHITECTURE

SAECOIN will use a decentralized P2P communication model.

Network Communication



Each node may connect to multiple independent peers.

P2P Features

- Peer discovery.

- Peer connection management.
- Message validation.
- Block announcement.
- Transaction announcement.
- Block download.
- Peer scoring.
- Connection management.
- Protocol version checking.

Network Security

The network must protect against:

- Malformed messages.
- Excessive connections.
- Spam transactions.
- Invalid blocks.
- Malicious peer behavior.
- Resource exhaustion.

A node must not depend on a single seed server for continued operation.

Seed nodes may assist initial discovery, but the network must remain decentralized.

---

6. BLOCKCHAIN SYNCHRONIZATION

New nodes must be able to synchronize with the network.

Synchronization Process

1. Start Node
   |
2. Load Local Database
   |
3. Discover Peers
   |
4. Connect to Peers
   |
5. Exchange Network Information
   |
6. Discover Best Available Chain
   |
7. Download Required Blocks
   |
8. Validate Blocks

|
9. Update UTXO State
|
10. Synchronization Complete

Synchronization Requirements

- Validate downloaded blocks.
- Track chain work.
- Detect missing blocks.
- Recover from interrupted downloads.
- Handle alternative chains.
- Support chain reorganization.
- Resume synchronization after restart.

The node must not blindly trust a peer's reported block height or chain state.

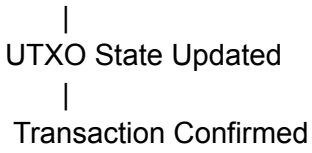
---

7. TRANSACTION PROCESSING ARCHITECTURE

SAECOIN uses the UTXO transaction model.

Transaction Flow

WALLET
|
Create Transaction
|
Select UTXOs
|
Sign Transaction
|
Broadcast to Node
|
Transaction Validation
|
MEMPOOL
|
Miner Selects
|
Block Created
|
Block Validation



Transaction Validation

Nodes must verify:

- Transaction structure.
- Input references.
- UTXO existence.
- Digital signatures.
- Input and output amounts.
- Transaction fees.
- Double-spend conditions.
- Locking and unlocking conditions.

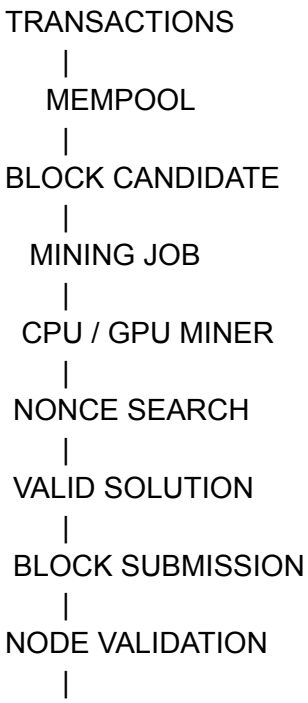
The exact validation rules will be finalized in the Consensus Specification.

---

8. CONSENSUS ARCHITECTURE

SAECOIN will use Proof of Work.

Proposed Consensus Flow



```
ACCEPT OR REJECT
      |
BLOCKCHAIN UPDATE
```

Consensus Requirements

- Deterministic block validation.
- Proof-of-Work verification.
- Difficulty adjustment.
- Block reward enforcement.
- Transaction validation.
- Chain selection.
- Reorganization support.

The proposed maximum supply is 21,000,000 SAC.

The exact issuance and subsidy rules must be finalized in `Consensus_Spec.md`.

---

9. MINING ARCHITECTURE

Mining will support CPU and GPU implementations.

CPU Mining

The CPU miner will:

- Receive mining jobs.
- Search for valid nonces.
- Calculate candidate hashes.
- Submit valid solutions.
- Display hashrate.

GPU Mining

The GPU miner will:

- Receive mining jobs.
- Process mining workloads on supported GPUs.
- Search for valid solutions.
- Submit candidate solutions.
- Report performance statistics.

GPU support depends on the final PoW algorithm and available hardware backends.

Mining Modes

Solo Mining

The miner works with its own full node and receives the block reward if it successfully mines a valid block.

Pool Mining

Multiple miners contribute work through mining-pool software. Pool payout rules are separate from blockchain consensus.

---

10. WALLET ARCHITECTURE

Wallets will use a shared wallet core where practical.

Android Wallet

Technology:

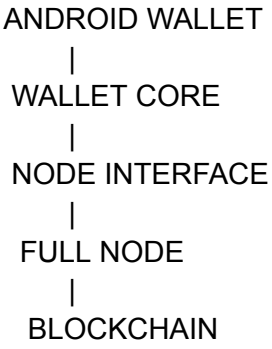
- Kotlin.
- Jetpack Compose.
- Rust wallet core through a suitable integration layer, if needed.

Windows Wallet

Technology:

- Rust.
- Tauri.
- TypeScript.

Wallet Communication



Wallet Responsibilities

- Create wallet.
- Restore wallet.
- Generate addresses.
- Display balance.
- Create transactions.
- Sign transactions locally.
- Broadcast transactions.
- Display transaction history.

Private keys must remain under wallet control.

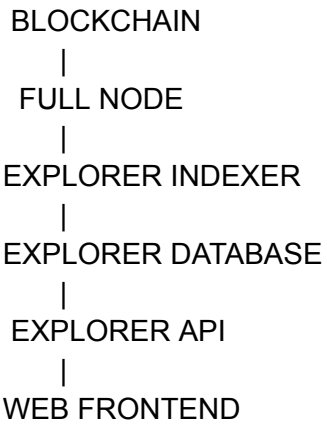
A remote node may provide blockchain information, but it must never receive the wallet's private keys.

---

11. BLOCK EXPLORER ARCHITECTURE

The explorer provides public blockchain information.

Explorer Architecture



Explorer Features

- Latest blocks.
- Block details.
- Transaction details.
- Address information.
- Network hashrate.
- Mining difficulty.
- Total supply statistics.
- Blockchain height.

Explorer indexing is not part of consensus.

If explorer data becomes corrupted, it must be possible to rebuild it from validated blockchain data.

---

12. API ARCHITECTURE

The node should expose a controlled interface for applications.

Possible API categories:

Node API

- Network status.
- Blockchain height.
- Node information.
- Peer information.

Wallet API

- Transaction broadcast.
- UTXO queries.
- Address-related blockchain queries.

Explorer API

- Block lookup.
- Transaction lookup.
- Address lookup.
- Network statistics.

Mining API

- Mining job information.
- Block submission.
- Mining status.

API transport and authentication requirements must be finalized in `API_Spec.md`.

Sensitive wallet operations should remain local to the wallet application.

---

13. NETWORK ENVIRONMENTS

SAECOIN should support separate network environments.

Development Network

Used by developers.

- Rapid testing.
- Experimental parameters.
- Local nodes.
- Test mining.

Testnet

Used for public testing.

- Independent testnet genesis block.
- Test coins.
- Multiple public nodes.
- Wallet testing.
- Mining testing.
- Explorer testing.

Mainnet

The permanent production network.

- Final genesis block.
- Final consensus parameters.
- Real SAC transactions.
- Public node participation.
- Production wallet support.

Development, testnet, and mainnet must use distinct network identifiers and address prefixes.

A transaction or block from one network must not be accidentally accepted as valid on another network.

---

14. GENESIS BLOCK ARCHITECTURE

The genesis block is the first block of the SAECOIN blockchain.

It must be defined before network launch.

Genesis configuration must specify:

- Network identifier.
- Genesis timestamp.
- Initial block parameters.
- Initial difficulty.
- Genesis block contents.
- Genesis block hash.
- Initial chain configuration.

All compatible nodes must independently recognize the same valid genesis block for their network.

The final genesis parameters must be documented and verified before public testnet or mainnet launch.

---

15. SECURITY ARCHITECTURE

Security must be implemented across all modules.

Blockchain Security

- Strict block validation.
- Strict transaction validation.
- Double-spend prevention.
- Correct PoW verification.
- Safe chain reorganization.

Network Security

- Peer limits.
- Message size limits.
- Malformed message rejection.
- Connection abuse protection.
- Rate limiting where appropriate.

Wallet Security

- Local signing.
- Encrypted key storage.
- Secure backup.
- Recovery verification.
- No private-key transmission.

Database Security

- Atomic state updates.

- Corruption detection.
- Safe recovery.
- Database backups where appropriate.

---

16. FAILURE RECOVERY

The system must handle:

- Unexpected node shutdown.
- Database interruption.
- Network disconnection.
- Interrupted synchronization.
- Invalid peer data.
- Corrupted local indexes.
- Chain reorganization.

The node must recover safely without silently accepting an invalid chain state.

Explorer indexes should be rebuildable.

Wallet backups must support recovery on another compatible device.

---

17. SCALABILITY ARCHITECTURE

SAECOIN should be designed for future network growth.

Potential improvements:

- More efficient block synchronization.
- Improved UTXO database access.
- Parallel transaction verification where safe.
- Efficient peer management.
- Explorer indexing optimization.
- Mining backend optimization.

Scalability improvements must not change consensus behavior unless introduced through a properly specified and coordinated protocol upgrade.

---

18. PROPOSED SOURCE CODE MODULES

```
saecoin/
│
├── crates/
│   ├── crypto/
│   ├── primitives/
│   ├── consensus/
│   ├── transactions/
│   ├── utxo/
│   ├── storage/
│   ├── p2p/
│   ├── node/
│   ├── miner/
│   ├── wallet-core/
│   └── api/
│
├── apps/
│   ├── android-wallet/
│   ├── desktop-wallet/
│   └── explorer/
│
├── services/
│   ├── explorer-indexer/
│   └── mining-pool/
│
├── tests/
│
├── docs/
│
└── README.md
```

This is a proposed modular structure. Final package boundaries may change during implementation.

---

19. DEVELOPMENT ORDER

Recommended implementation order:

1. Cryptographic primitives.
2. Core data structures.

3. Transaction serialization.
4. UTXO engine.
5. Block structure.
6. Proof-of-Work implementation.
7. Difficulty adjustment.
8. Genesis block.
9. Local blockchain storage.
10. Single-node operation.
11. P2P networking.
12. Multi-node synchronization.
13. CPU mining.
14. GPU mining implementation.
15. Wallet core.
16. Android wallet.
17. Windows wallet.
18. Explorer.
19. Public testnet.
20. Security and performance audits.

The project must not skip foundational consensus testing.

---

20. FINAL ARCHITECTURE PRINCIPLES

1. SAECOIN must be an independent blockchain.
2. Nodes must independently validate consensus.
3. No central server should control transaction validity.
4. Wallet private keys must remain private.
5. Blockchain monetary calculations must use integer arithmetic.
6. Consensus data must be deterministic.
7. Explorer and pool databases must remain separate from consensus state.
8. All network messages must be validated.
9. Chain reorganizations must be handled safely.
10. Mainnet must not launch before extensive testing.

---

21. FINAL NOTES

This document defines the proposed overall architecture of SAECOIN.

It does not finalize:

- The PoW mining algorithm.

- Cryptographic hash selection.
- Digital signature algorithm.
- Exact binary serialization.
- Difficulty adjustment formula.
- Block size and transaction limits.
- P2P wire protocol.
- Final API specifications.

These decisions must be completed in their dedicated technical documents.

All current supply and consensus parameters remain provisional.

END OF Blockchain_Architecture.md